

Homework 2, Convex Optimization

Departamento de Matemáticas, Universidad de los Andes
Colombia

Student's name: Luis Alejandro Rubiano
Student's code: 202013482
Email address: la.rubiano@uniandes.edu.co
Class: Convex Optimization
Instructor: Mauricio Junca

1 Find the minimum over $\mathbb{R}_{++}^2$ of the function $f(x, y) = \frac{1}{xy} + x + y$.

Solution: The function $\frac{1}{xy}$ has hessian matrix given by $\begin{pmatrix} \frac{2}{x^3y} & \frac{1}{x^2y^2} \\ \frac{1}{x^2y^2} & \frac{2}{xy^3} \end{pmatrix}$.

Then let $a, b \in \mathbb{R}$, let $v = \frac{a}{x^{3/2}y^{1/2}}$ and $w = \frac{b}{x^{1/2}y^{3/2}}$, then for $x, y \geq 0$ we have that

$$\begin{pmatrix} a & b \end{pmatrix} \begin{pmatrix} \frac{2}{x^3y} & \frac{1}{x^2y^2} \\ \frac{1}{x^2y^2} & \frac{2}{xy^3} \end{pmatrix} \begin{pmatrix} a \\ b \end{pmatrix} = 2 \left(\frac{a^2}{x^3y} + \frac{b^2}{xy^3} + \frac{ab}{x^2y^2} \right) \\ = 2(v^2 + w^2 + vw)$$

Note that this last expression can only be negative if either v or w is negative, but this is not possible since w.l.o.g. assume $v < 0 < w$ then

- Case 1: if $-v \leq w$ then $-vw \leq w^2$ or $0 \leq w^2 + vw$.
- Case 2: if $w < -v$ then $-vw < v^2$ or $0 < v^2 + vw$.

Therefore, $2 \left(\frac{a^2}{x^3y} + \frac{b^2}{xy^3} + \frac{ab}{x^2y^2} \right) \geq 0$ for all $a, b \in \mathbb{R}$ and $x, y \geq 0$ and the hessian matrix is positive semidefinite.

Then the function $f(x, y) = \frac{1}{xy} + x + y$ is convex over $\mathbb{R}_{++}^2$ since its the sum of convex functions.

Therefore the minimum of $f(x, y)$ over $\mathbb{R}_{++}^2$ is the point where the gradient of $f(x, y)$ is zero.

The gradient of $f(x, y)$ is given by $\nabla f(x, y) = \begin{pmatrix} -\frac{1}{x^2y} + 1 \\ -\frac{1}{xy^2} + 1 \end{pmatrix}$, setting it equal to zero we get that $x = y = 1$ is

the only point where the gradient is zero, and since the hessian is positive semidefinite, then $x = y = 1$ is the minimum of $f(x, y)$ over $\mathbb{R}_{++}^2$.

2 Consider the function $f(x, y) = x^2 - \alpha xy^2 + 2y^4$. Show that for all parameter values except two, the origin $(0, 0)$ is the only critical point of f . Ex. 2.9 Foundations of Optimization, Güller

- (a) Find the exceptional α 's and show that f has infinitely many critical points for these α values. Determine the nature of these critical points.

Solution: Note that f is a polynomial in two variables, thus when calculating the directional derivative of f at a point (x_0, y_0) in the direction of a vector (d_1, d_2) we have that $f(x_0 + td_1, y_0 + td_2) - f(x_0, y_0)$ will also be a polynomial in t , thus when dividing by t and taking the limit as t goes to zero, the higher order terms will vanish and we will be left with only the linear terms, which implies that the directional derivative of f at (x_0, y_0) in the direction of (d_1, d_2) , exists and is a linear.

Then the gradient of f is given by $\nabla f(x, y) = \begin{pmatrix} 2x - \alpha y^2 \\ -2\alpha xy + 8y^3 \end{pmatrix}$, setting it equal to zero we get:

$$2x - \alpha y^2 = 0 \quad (1)$$

$$-2\alpha xy + 8y^3 = 0 \quad (2)$$

$$\Rightarrow x = (\alpha y^2)/2$$

Replacing this into the second equation we get that

$$-2\alpha y(\alpha y^2)/2 + 8y^3 = 0$$

$$-\alpha^2 y^3 + 8y^3 = 0$$

$$(8 - \alpha^2)y^3 = 0$$

Then, the only way for this equation to hold is if $y = 0$ or $8 - \alpha^2 = 0$, the first case implies that $x = 0$ and the second case implies that $\alpha = \pm 2\sqrt{2}$.

When $\alpha = 2\sqrt{2}$, setting the gradient of f equal to zero we get:

$$2x - 2\sqrt{2}y^2 = 0 \quad (1)$$

$$-4\sqrt{2}xy + 8y^3 = 0 \quad (2)$$

Setting the two equations, equal to each other, we get that the set of solutions is given by the parabola $x = \sqrt{2}y^2$, which has infinitely many points.

In an analogous way, when $\alpha = -2\sqrt{2}$, setting the gradient of f equal to zero we get that the set of solutions is given by the parabola $x = -\sqrt{2}y^2$, which has infinitely many points.

To see the nature of the critical points, we can see that by the same argument as before, each entry of $\nabla f(x, y)$ is a polynomial in x and y , thus the hessian of f is well defined.

Then, the hessian of f , with $\alpha = 2\sqrt{2}$ is given by

$$H_f(x, y) = \begin{pmatrix} 2 & -2\sqrt{2}y \\ -2\sqrt{2}y & -8\sqrt{2}x + 24y^2 \end{pmatrix}$$

Evaluating at the parabola, using the substitution $x := \sqrt{2}y^2$ we get that

$$H_f(\sqrt{2}y^2, y) = \begin{pmatrix} 2 & -2\sqrt{2}y \\ -2\sqrt{2}y & 32y^2 \end{pmatrix}$$

This matrix has eigenvalues $1 + 16y^2 - \sqrt{1 + 256y^4}$ and $1 + 16y^2 + \sqrt{1 + 256y^4}$, which are both positive for all $y \neq 0$, thus the hessian is positive definite except at the origin, where it is positive semidefinite. When $y \neq 0$ the critical points are strict local minimizers. The case for $\alpha = -2\sqrt{2}$ is analogous by symmetry.

- (b) Consider the values of α 's for which the origin is the only critical point. For each α , determine the nature of the critical point. Show that in some cases, the origin is a local minimum, but in other cases, it is a saddle point.

Solution: We claim the following: When $-2\sqrt{2} < \alpha < 2\sqrt{2}$, the origin is a global minimum. To see this, we can complete the square and see $f(x, y) = x^2 - \alpha xy^2 + 2y^4 = x^2 + y^2(2y^2 - \alpha x) = \left(x - \frac{\alpha y^2}{2}\right)^2 + \left(2 - \frac{\alpha^2}{4}\right)y^4$.

The first term is squared, thus always greater or equal to zero, and the second term is positive when $-2\sqrt{2} < \alpha < 2\sqrt{2}$, thus since $f(0, 0) = 0$, and $f(x, y) > 0$ for all $(x, y) \neq (0, 0)$, then the origin is a global minimum.

When $\alpha \in \mathbb{R} \setminus [-2\sqrt{2}, 2\sqrt{2}]$, the origin is a saddle point. W.L.O.G assume $\alpha > 2\sqrt{2}$, the other case is completely analogous. To see this, in the previous point we already showed the origin it is a critical point, we can now take $\varepsilon > 0$. Then, $f\left(\frac{\alpha\varepsilon^2}{2}, \varepsilon\right) = -\frac{\alpha^2\varepsilon^4}{4} + 2\varepsilon^4 = \varepsilon^4\left(2 - \frac{\alpha^2}{4}\right) < 0$, but moving along the x axis, we have that $f(\varepsilon, 0) = \varepsilon^2 > 0$, thus by definition the origin is a saddle point.

- (c) Show that even when the origin is a saddle point, $(0, 0)$ is a local strict minimizer of f on every line passing through the origin. In fact, show that, except for one line, the function $g(t) := f(td)$ satisfies $g'(0) = 0$ and $g''(0) > 0$.

Solution: Note that when substituting $g(t) := f(td) = (td_1)^2 - \alpha(td_1)(td_2)^2 + 2(td_2)^4$,
We get that $g'(t) = 2td_1^2 - 3\alpha d_1 d_2^2 t^2 + 8d_2^4 t^3$ and $g''(t) = 2d_1^2 - 6\alpha d_1 d_2^2 t + 24d_2^4 t^2$.

Then, $g'(0) = 0$ and $g''(0) = 2d_1^2 > 0$, except for the line $d_1 = 0$, where $g''(0) = 0$.

3 Program the steepest descent and Newton algorithms using the backtracking line search, Algorithm 3.1. Use them to minimize the Rosenbrock function (2.22). Set the initial step length $\alpha_0 = 1$, and print the step length used by each method at each iteration. First try the initial point $x_0 = (1.2, 1.2)^T$ and then the more difficult starting point $x_0 = (-1.2, 1)^T$. **Ex. 3.1. Numerical Optimization, Nocedal & Wright**

Solution: Python3 code to solve this problem is given by:

```

1  import sympy as sp
2  import matplotlib.pyplot as plt
3  import os
4
5  x_1, x_2 = sp.symbols('x_1 x_2')
6
7  f = 100 * (x_2 - x_1**2)**2 + (1 - x_1)**2
8
9
10 def steepest_descent(x_k, gradient, hessian):
11     # Returns the gradient at x_k
12     return - gradient.subs({x_1: x_k[0], x_2: x_k[1]})
13
14
15 def newton(x_k, gradient, hessian):
16     # Returns - H^-1 evaluated at x_k times the gradient at x_k
17     return - hessian.subs({x_1: x_k[0], x_2: x_k[1]}) .inv() * \
18         gradient.subs({x_1: x_k[0], x_2: x_k[1]})
19
20
21 def graph(x, y, title, x_label, y_label, filename):
22     fig = plt.figure()
23     plt.plot(x, y)
24     plt.title(title)
25     plt.xlabel(x_label)

```

```

26 plt.gca().axis.set_major_locator(plt.MaxNLocator(integer=True))
27 plt.ylabel(y_label)
28 filename = "Tarea2" + os.sep + "images" + os.sep + filename + ".png"
29 fig.savefig(filename, dpi=fig.dpi)
30 plt.clf()
31
32
33 def minimize(f, x_0, method, a_0):
34     rho = 0.5
35     c = 0.5
36     iterations = 0
37     max_iterations = 20
38
39     gradient = sp.Matrix([f.diff(x_1), f.diff(x_2)])
40     hessian = sp.hessian(f, (x_1, x_2))
41
42     if method == 'steepest_descent':
43         p = steepest_descent
44     elif method == 'newton':
45         p = newton
46
47     X = [x_0]
48     F = [f.subs({x_1: x_0[0], x_2: x_0[1]})]
49     A = [a_0]
50     P = []
51
52     P.append(p(X[-1], gradient, hessian))
53
54     while iterations < max_iterations:
55         while f.subs({x_1: X[-1][0] + A[-1] * P[-1][0],
56                     x_2: X[-1][1] + A[-1] * P[-1][1]}) > f.subs({x_1: X[-1][0],
57                                                                 x_2: X[-1][1]}) + c * A
58 [-1] * gradient.subs({x_1: X[-1][0],
59                     x_2: X[-1][1]}) * P[-1]):
60             A[-1] = rho * A[-1]
61
62             X.append(X[-1] + A[-1] * P[-1])
63             F.append(f.subs({x_1: X[-1][0], x_2: X[-1][1]}))
64             P.append(p(X[-1], gradient, hessian))
65             A.append(a_0)
66
67             iterations += 1
68
69     return X, F, A[:-1], P
70
71 X, F, A, P = minimize(f, x_0=sp.Matrix(
72     [1.2, 1.2]), method='steepest_descent', a_0=1)
73
74 graph([i for i in range(len(A))],
75     A,
76     r"Step size, using Steepest Descent, $x_0 = [1.2, 1.2]$",
77     "Iteration",
78     r"$\alpha$",
79     "steepest_descent_step_size_a")
80 graph([i for i in range(len(F))],
81     F,
82     r"$f(x)$, using Steepest Descent, $x_0 = [1.2, 1.2]$",
83     "Iteration",
84     r"$f(x)$",
85     "steepest_descent_fx_a")
86
87 X, F, A, P = minimize(f, x_0=sp.Matrix([1.2, 1.2]), method='newton', a_0=1)
88
89 graph([i for i in range(len(A))],
90     A,
91     r"Step size, using Newton, $x_0 = [1.2, 1.2]$",
92     "Iteration",
93     r"$\alpha$",
94     "newton_step_size_a")
95 graph([i for i in range(len(F))],
96     F,
97     r"$f(x)$, using Newton, $x_0 = [1.2, 1.2]$",
98     "Iteration",

```

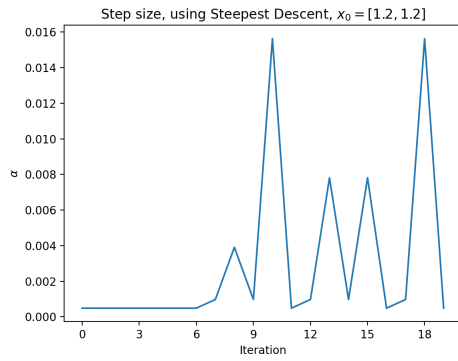


Figure 1: Step size, using Steepest Descent, $x_0 = [1.2, 1.2]$

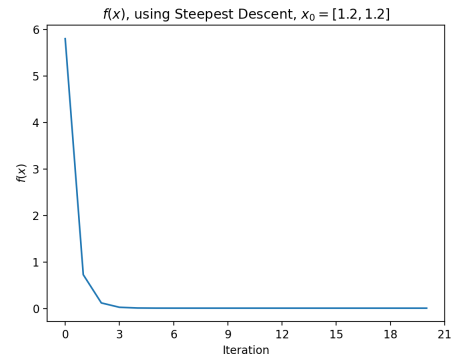


Figure 2: $f(x)$, using Steepest Descent, $x_0 = [1.2, 1.2]$

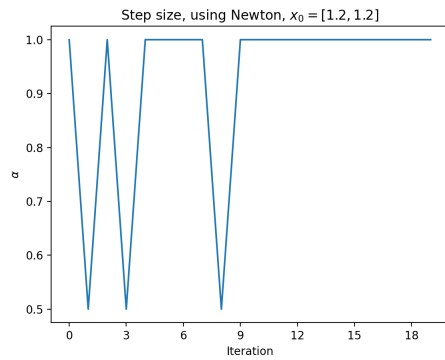


Figure 3: Step size, using Newton, $x_0 = [1.2, 1.2]$

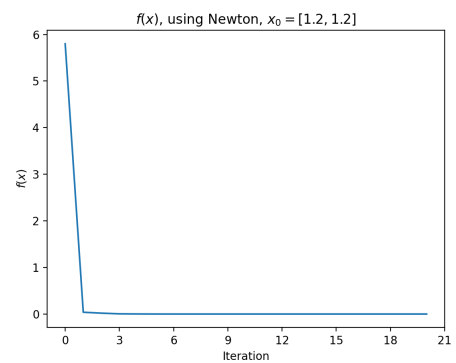


Figure 4: $f(x)$, using Newton, $x_0 = [1.2, 1.2]$

```

99         r"$f(x)$",
100         "newton_fx_a")
101
102     X, F, A, P = minimize(f, x_0=sp.Matrix(
103         [-1.2, 1]), method='steepest_descent', a_0=1)
104
105     graph([i for i in range(len(A))],
106           A,
107           r"Step size, using Steepest Descent, $x_0 = [-1.2, 1]$",
108           "Iteration",
109           r"$\alpha$",
110           "steepest_descent_step_size_b")
111     graph([i for i in range(len(F))],
112           F,
113           r"$f(x)$, using Steepest Descent, $x_0 = [-1.2, 1]$",
114           "Iteration",
115           r"$f(x)$",
116           "steepest_descent_fx_b")
117
118     X, F, A, P = minimize(f, x_0=sp.Matrix([-1.2, 1]), method='newton', a_0=1)
119
120     graph([i for i in range(len(A))],
121           A,
122           r"Step size, using Newton, $x_0 = [-1.2, 1]$",
123           "Iteration",
124           r"$\alpha$",
125           "newton_step_size_b")
126     graph([i for i in range(len(F))],
127           F,
128           r"$f(x)$, using Newton, $x_0 = [-1.2, 1]$",
129           "Iteration",
130           r"$f(x)$",
131           "newton_fx_b")

```

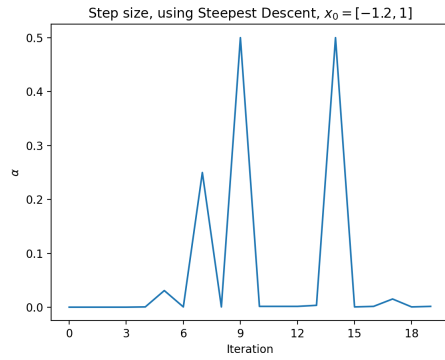


Figure 5: Step size, using Steepest Descent, $x_0 = [-1.2, 1]$

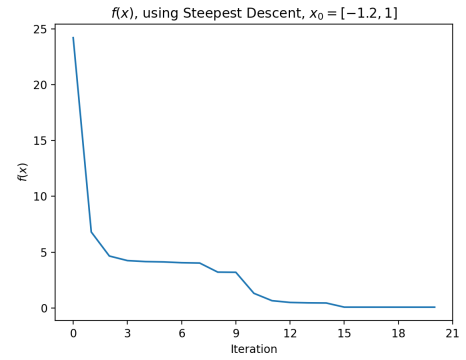


Figure 6: $f(x)$, using Steepest Descent, $x_0 = [-1.2, 1]$

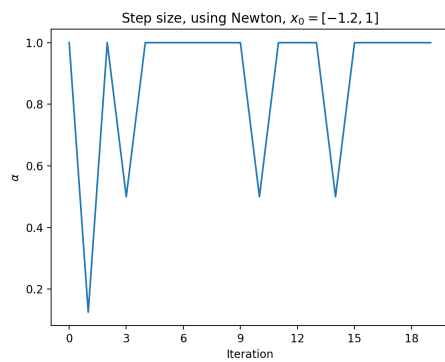


Figure 7: Step size, using Newton, $x_0 = [-1.2, 1]$

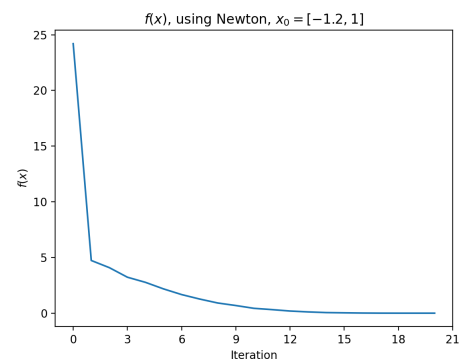


Figure 8: $f(x)$, using Newton, $x_0 = [-1.2, 1]$

5 Consider the function $f(x) = \langle c, x \rangle + \sum_{j=1}^m \log(1 - \langle a_j, x \rangle) - \sum_{i=1}^n \log(1 - x_i^2)$.

Take $n = 5000$ and $m = 2000$, and vectors c, a_j chosen arbitrarily in such a way that $\|a_j\| = 1$. For each of the methods, start the iteration at $x = 0$ and stop the iteration when $\|\nabla f(x)\| < \varepsilon$. Graph $\log(f(x_k) - f(x^*))$ in each iteration, (estimate $f(x^*)$) as best as you can and compare the time of execution of the methods.

In all the problems it was chosen $c_i = 1$ for all i . In non of the problems it was possible to use $\varepsilon = 10^{-6}$, since the gradient of the function was always greater than this value, possible due to accumulation of errors. Six iterations of each method were used.

The vectors a_j were generated using the following code:

```
1 import sympy as sp
2 import sympy.stats as stats
3
4 Z = stats.Normal('Z', 0, 1)
5
6 n = 5000
7 m = 2000
8 A = []
9
10 for i in range(m):
11     Aj = sp.Matrix(stats.sample(Z, size=n, seed=3527+i))
12     norm_Aj = sp.sqrt(sum([a**2 for a in Aj]))
13     Aj = (1 / norm_Aj) * Aj # Normalize Aj
14     A.append(Aj)
15     if i % 100 == 0:
16         print(f'{i} vectors generated')
17
18 # Save A to a file, where each row is a vector Aj
19 with open('A.txt', 'w') as f:
20     for Aj in A:
21         f.write(','.join([str(a) for a in Aj]) + '\n')
```

- (a) Use the gradient method with a fixed step length step, indicating the value of step used and why you chose it.

Solution: In this case we used $\alpha = 0.2$ as the step size, since it is a small value that allowed the problem to work without getting out of the domain of the logarithm function. The execution time of this method was 1.1878 hours.

The code used to solve this problem is given by:

```
1 import matplotlib.pyplot as plt
2 import os
3 from time import perf_counter
4 from numpy import float16, dot, ones, zeros, log
5 from numpy.linalg import norm, solve
6 import pandas as pd
7 from progress.bar import Bar
8
9 n = 5000
10 m = 2000
11 epsilon = float16(1e-6)
12 max_iterations = 6 # Maximum number of iterations
13 max_sub_iterations = 3 # Maximum number of sub-iterations
14 # C is the vector of ones
15 ONE = float16(1)
16 TWO = float16(2)
17
18 dots = {}
19 def optimized_dot(x,y):
20     stringxy = str(x) + str(y)
```

```

21     if stringx in dots:
22         return dots[stringx]
23     result = dot(x,y)
24     dots[stringx] = result
25     return result
26
27 fs = {}
28 def f(x, A):
29     stringx = str(x)
30     if stringx in fs:
31         return fs[stringx]
32
33     result = sum(x)
34     for j in range(m):
35         result -= log(ONE - optimized_dot(A[j],x))
36     for i in range(n):
37         result -= log(ONE - x[i]*x[i])
38
39     fs[stringx] = result
40     return result
41
42 gradients = {}
43 def gradient(x, A):
44     stringx = str(x)
45     if stringx in gradients:
46         return gradients[stringx]
47
48     result = ones(n, dtype=float16)
49     with Bar('Gradient', fill='#', suffix='%(percent).1f%% - %(eta)ds', max=n) as bar:
50         for i in range(n):
51             for j in range(m):
52                 result[i] += A[j][i] / (ONE - optimized_dot(A[j],x))
53                 result[i] += TWO * x[i] / (ONE - x[i]*x[i])
54             bar.next()
55
56     gradients[stringx] = result
57     return result
58
59 hessians = {}
60 def hessian(x, A):
61     # We assume its symmetric
62     stringx = str(x)
63     if stringx in hessians:
64         return hessians[stringx]
65
66     result = zeros((n,n), dtype=float16)
67     with Bar('Hessian', fill='#', suffix='%(percent).1f%% - %(eta)ds', max=n*(n+1)/2) as
bar:
68         for i in range(n):
69             for l in range(i,n):
70                 for j in range(m):
71                     result[i][l] += A[j][i] * A[j][l] / ((ONE - optimized_dot(A[j],x)) * (
ONE - optimized_dot(A[j],x)))
72                     if i == l:
73                         result[i][l] += TWO* (ONE + x[i]*x[i]) / ((ONE - x[i]*x[i]) * (ONE - x
[i]*x[i]))
74
75                     result[l][i] = result[i][l]
76                 bar.next()
77
78     hessians[stringx] = result
79     return result
80
81
82 def graph(x, y, title, x_label, y_label, filename):
83     fig = plt.figure()
84     plt.plot(x, y)
85     plt.title(title)
86     plt.xlabel(x_label)
87     plt.gca().xaxis.set_major_locator(plt.MaxNLocator(integer=True))
88     plt.ylabel(y_label)
89     filename = "images" + os.sep + filename + ".png"
90     fig.savefig(filename, dpi=fig.dpi)
91     plt.clf()
92

```



```

93 A = []
94 df = pd.read_csv('A.txt', header=None)
95 data = df.values
96 for j in range(m):
97     # Save data[j] as numpy array of float16
98     A.append(data[j].astype(float16))
99 print("A loaded")
100
101
102 time0 = perf_counter()
103
104 ## Gradient method with constant step size alpha
105
106 alpha = float16(0.2) # Small constant alpha
107 X = [zeros(n, dtype=float16)]
108 Fx = [f(X[-1], A)]
109 P = [- gradient(X[-1], A)]
110 gradient_norms = [norm(P[-1])]
111
112 iterations = 0
113 while (gradient_norms[-1] > epsilon) and (iterations < max_iterations):
114     print("Iteration", iterations)
115     print("Norm of gradient:", gradient_norms[-1])
116     print("f(x_k):", Fx[-1])
117     print("x_k:", X[-1])
118     iterations += 1
119     X.append(X[-1] + alpha * P[-1]) # X_{k+1} = X_k + alpha * P_k
120     Fx.append(f(X[-1], A))
121     P.append(- gradient(X[-1], A))
122     gradient_norms.append(norm(P[-1]))
123
124
125 # End timer
126 time1 = perf_counter()
127
128
129 print("Gradient method with constant step size alpha =", alpha, "completed")
130 print("Iterations:", iterations)
131 print("Total time elapsed:", (time1 - time0)/3600, "hours")
132
133 graph(range(iterations+1), Fx, "f(x) vs iterations", "Iteration", "f(x)", "5ai")
134 graph(range(iterations+1), gradient_norms, "Norm of gradient vs iterations", "Iteration",
"Norm of gradient", "5aai")

```

(b) Use the gradient method with *backtracking*.

Solution: Here we used the backtracking method with a initial alpha of 0.25, $\rho = 0.5$ and $c = 0.5$. The execution time of this method was 1.0337 hours. The code used to solve this problem is given by:

```

1 alpha = float16(0.25) # Initial alpha
2 rho = float16(0.5)
3 c = float16(0.5)
4 X = [zeros(n, dtype=float16)]
5 Fx = [f(X[-1], A)]
6 ALPHAS = [alpha]
7 P = [- gradient(X[-1], A)]
8 gradient_norms = [norm(P[-1])]
9 iterations = 0
10
11 while (gradient_norms[-1] > epsilon) and (iterations < max_iterations):
12     print("Iteration", iterations)
13     print("Norm of gradient:", gradient_norms[-1])
14     print("f(x_k):", Fx[-1])
15     print("x_k:", X[-1])
16     print("Alpha:", ALPHAS[-1])
17
18     iterations += 1
19     sub_iterations = 0
20
21     while f(X[-1] + ALPHAS[-1] * P[-1], A) > f(X[-1], A) + c * ALPHAS[-1] * optimized_dot(
gradient(X[-1], A), P[-1]) and sub_iterations < max_sub_iterations:
22         sub_iterations += 1
23         ALPHAS[-1] = rho * ALPHAS[-1]
24

```

```

25     X.append(X[-1] + ALPHAS[-1] * P[-1]) #  $X_{k+1} = X_k + \alpha * P_k$ 
26     Fx.append(f(X[-1], A))
27     P.append(- gradient(X[-1], A))
28     gradient_norms.append(norm(P[-1]))
29     ALPHAS.append(alpha)
30
31 # End timer
32 time2 = perf_counter()
33
34 print("Gradient method with backtraking completed")
35 print("Iterations:", iterations)
36 print("Total time elapsed:", (time2 - time1)/3600, "hours")
37
38 graph(range(iterations+1), Fx, "f(x) vs iterations", "Iteration", "f(x)", "5bi")
39 graph(range(iterations+1), gradient_norms, "Norm of gradient vs iterations", "Iteration", "
    Norm of gradient", "5bii")
40 graph(range(iterations+1), ALPHAS, "Alpha vs iterations", "Iteration", "Alpha", "5biii")

```

(c) Use the gradient method with *backtracking* and Goldstein or Wolfe conditions.

Solution: The initial alpha was set to 0.25, $c_1 = 10^{-4}$ and $c_2 = 0.9$. And we used the Wolfe conditions. The execution time of this method was 0.6928 hours. The code used to solve this problem is given by:

```

1     alpha = float16(0.25) # Initial alpha
2     c_1 = float16(10e-4)
3     c_2 = float16(0.9)
4     X = [zeros(n, dtype=float16)]
5     Fx = [f(X[-1], A)]
6     ALPHAS = [alpha]
7     P = [- gradient(X[-1], A)]
8     gradient_norms = [norm(P[-1])]
9     iterations = 0
10
11     while (gradient_norms[-1] > epsilon) and (iterations < max_iterations):
12         print("Iteration", iterations)
13         print("Norm of gradient:", gradient_norms[-1])
14         print("f(x_k):", Fx[-1])
15         print("x_k:", X[-1])
16         print("Alpha:", ALPHAS[-1])
17
18         iterations += 1
19         sub_iterations = 0
20
21         while (f(X[-1] + ALPHAS[-1] * P[-1], A) > f(X[-1], A) + c_1 * ALPHAS[-1] *
    optimized_dot(gradient(X[-1], A), P[-1]) or (- optimized_dot(gradient(X[-1] + ALPHAS[-1] *
    P[-1], A), P[-1]) > - c_2 * optimized_dot(gradient(X[-1], A), P[-1]))) and sub_iterations
    < max_sub_iterations:
22             sub_iterations += 1
23             ALPHAS[-1] = rho * ALPHAS[-1]
24
25         X.append(X[-1] + ALPHAS[-1] * P[-1]) #  $X_{k+1} = X_k + \alpha * P_k$ 
26         Fx.append(f(X[-1], A))
27         P.append(- gradient(X[-1], A))
28         gradient_norms.append(norm(P[-1]))
29         ALPHAS.append(alpha)
30
31 # End timer
32 time3 = perf_counter()
33
34 print("Gradient method with backtraking and Wolfe conditions completed")
35 print("Iterations:", iterations)
36 print("Total time elapsed:", (time3 - time2)/3600, "hours")
37
38 graph(range(iterations+1), Fx, "f(x) vs iterations", "Iteration", "f(x)", "5ci")
39 graph(range(iterations+1), gradient_norms, "Norm of gradient vs iterations", "Iteration",
    "Norm of gradient", "5cii")
40 graph(range(iterations+1), ALPHAS, "Alpha vs iterations", "Iteration", "Alpha", "5ciii")

```

(d) Use the Newton method with a fixed step length step.

Solution: We used a step size of 0.2 for the Newton method. The execution time of this method could not be calculated since each iteration takes more than 30 days, even after multiple optimizations. The code used to solve this problem is given by:

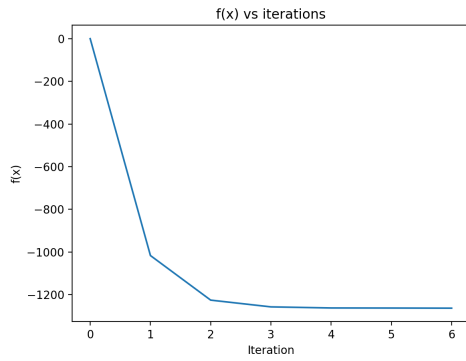


Figure 9: $f(x)$, using gradient method, $x_0 = 0$, $\alpha = 0.2$

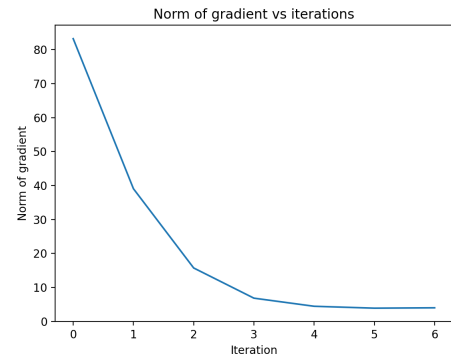


Figure 10: Norm of gradient, using gradient method, $x_0 = 0$, $\alpha = 0.2$

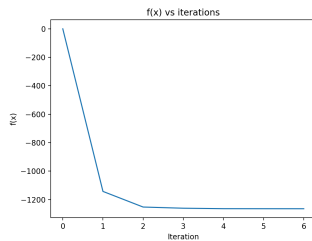


Figure 11: $f(x)$, using gradient method with backtracking, $x_0 = 0$, $\alpha = 0.25$, $\rho = 0.5$, $c = 0.5$

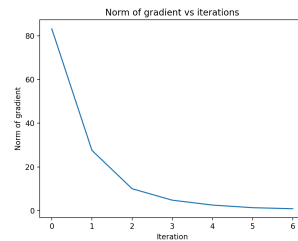


Figure 12: Norm of gradient, using gradient method with backtracking, $x_0 = 0$, $\alpha = 0.25$, $\rho = 0.5$, $c = 0.5$

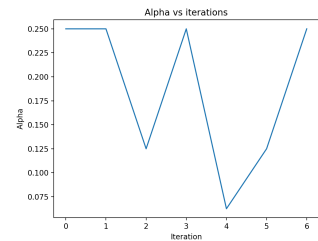


Figure 13: Alpha, using gradient method with backtracking, $x_0 = 0$, $\alpha = 0.25$, $\rho = 0.5$, $c = 0.5$

```

1 alpha = float16(0.2) # Small constant alpha
2 X = [zeros(n, dtype=float16)]
3 Fx = [f(X[-1], A)]
4 P = [solve(hessian(X[-1], A), -gradient(X[-1], A))]
5 iterations = 0
6
7 while iterations < max_iterations:
8     print("Iteration", iterations)
9     print("f(x_k):", Fx[-1])
10    print("x_k:", X[-1])
11    iterations += 1
12    X.append(X[-1] + alpha * P[-1]) # X_{k+1} = X_k + alpha * P_k
13    Fx.append(f(X[-1], A))
14    P.append(solve(hessian(X[-1], A), -gradient(X[-1], A)))
15
16 # End timer
17 time4 = perf_counter()
18
19 print("Newton method with constant step size alpha =", alpha, "completed")
20 print("Iterations:", iterations)
21 print("Total time elapsed:", (time4 - time3)/3600, "hours")
22
23 graph(range(iterations+1), Fx, "f(x) vs iterations", "Iteration", "f(x)", "5di")

```

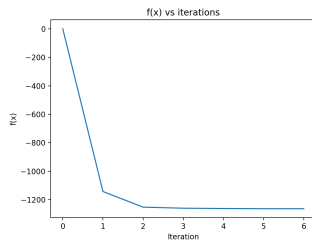


Figure 14: $f(x)$, using gradient method with backtracking and Wolfe conditions, $x_0 = 0$, $\alpha = 0.25$, $c_1 = 10^{-4}$, $c_2 = 0.9$

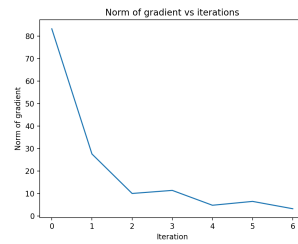


Figure 15: Norm of gradient, using gradient method with backtracking and Wolfe conditions, $x_0 = 0$, $\alpha = 0.25$, $c_1 = 10^{-4}$, $c_2 = 0.9$

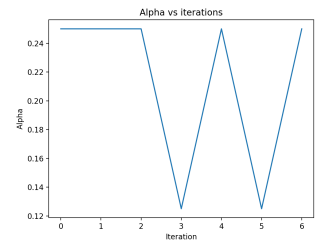


Figure 16: Alpha, using gradient method with backtracking and Wolfe conditions, $x_0 = 0$, $\alpha = 0.25$, $c_1 = 10^{-4}$, $c_2 = 0.9$